

Nang Ma Phyu

4CS-12

CST-4211:Parallel and Distributed Computing

Assignment-IV

1.Chat Application

Sever.java

Package chatsever

import java.io.*;

import java.net.*;

import java.util.Scanner;

public class Server {

public static void main(String[] args) {

try (ServerSocket serverSocket = new ServerSocket(5000)) {

System.out.println("Server listening on port 5000...");

Socket socket = serverSocket.accept();

System.out.println("Client connected!");

BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

Scanner scanner = new Scanner(System.in);

String clientMsg, serverMsg;

while (true) {

System.out.print("Server: ");

serverMsg = scanner.nextLine();

out.println(serverMsg);

if (serverMsg.equalsIgnoreCase("exit")) {

System.out.println("Chat ended by Server.");

break;

}

```

        clientMsg = in.readLine();
        if (clientMsg == null || clientMsg.equalsIgnoreCase("exit")) {
            System.out.println("Client disconnected/ended chat.");
            break;
        }
        System.out.println("Client: " + clientMsg);
    }
    socket.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Client.java

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 5000)) {
            System.out.println("Connected to Server!");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            String serverMsg, clientMsg;

            while (true) {

                serverMsg = in.readLine();
                if (serverMsg == null || serverMsg.equalsIgnoreCase("exit")) {
                    System.out.println("Server disconnected/ended chat.");
                    break;
                }
                System.out.println("Server: " + serverMsg);
            }
        }
    }
}

```

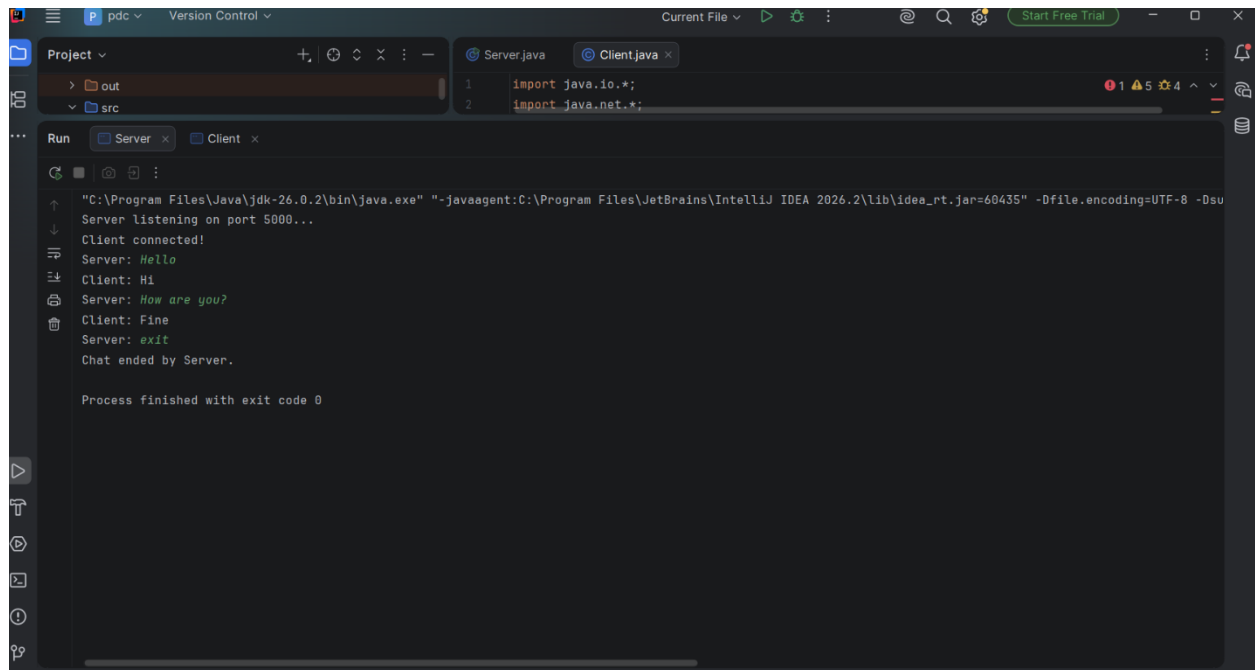
```

        System.out.print("Client: ");
        clientMsg = scanner.nextLine();
        out.println(clientMsg);

        if (clientMsg.equalsIgnoreCase("exit")) {
            System.out.println("Chat ended by Client.");
            break;
        }
    }
    socket.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Output



The screenshot shows the IntelliJ IDEA IDE with a project named 'pdc'. The 'Run' tab is active, displaying the output of a Java application. The application consists of two files: 'Server.java' and 'Client.java'. The 'Server.java' file shows the following code:

```

1 import java.io.*;
2 import java.net.*;

```

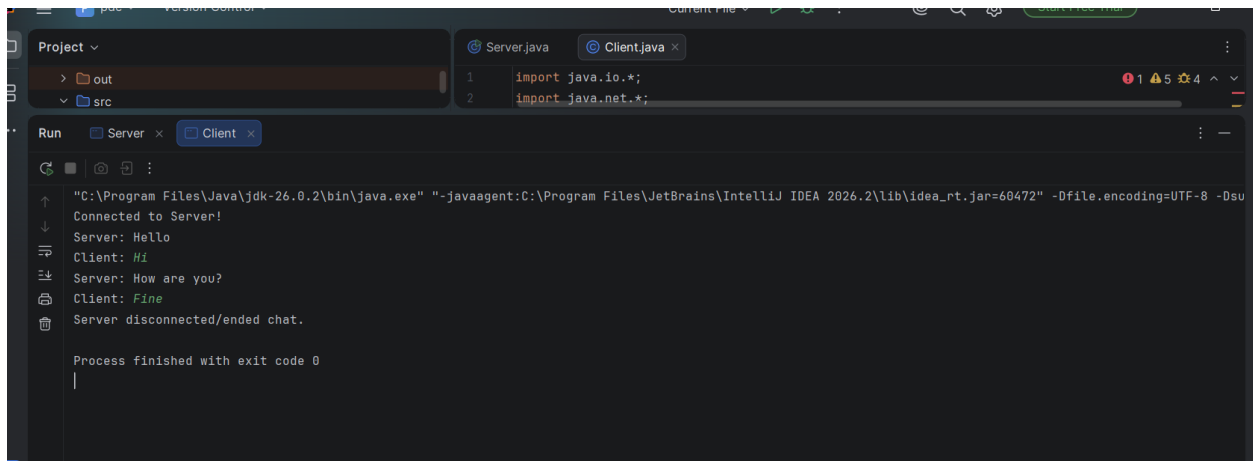
The 'Run' console shows the following output:

```

"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=60435" -Dfile.encoding=UTF-8 -Dsu
Server listening on port 5000...
Client connected!
Server: Hello
Client: Hi
Server: How are you?
Client: Fine
Server: exit
Chat ended by Server.

Process finished with exit code 0

```



2. Login Authentication

AuthServer.java

Package Auth

package Auth;

```
import java.io.*;
import java.net.*;
```

```
public class AuthServer {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {
            System.out.println("Authentication Server running on port 5000...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");

            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            String username = in.readLine();
            String password = in.readLine();

            System.out.println("Received authentication request for user: " + username);

            if ("admin".equals(username) && "12345".equals(password)) {
                out.println("Login Successful");
            } else {
```

```

        out.println("Invalid Username or Password");
    }

    socket.close();
    System.out.println("Server closed connection.");
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

AuthClinet.java

Package Auth

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class AuthClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 5000)) {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            System.out.print("Enter Username: ");
            String username = scanner.nextLine();

            System.out.print("Enter Password: ");
            String password = scanner.nextLine();

            out.println(username);
            out.println(password);

            String response = in.readLine();
            System.out.println("Server Response: " + response);

        } catch (IOException e) {

```

```

        e.printStackTrace();
    }
}
}

```

Output

```

Run
AuthClient x AuthServer x
"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=58851" -Dfile.encoding=UTF-8 -Dsu
Authentication Server running on port 5000...
Client connected!
Status: Login Successful
Server closed connection.
Process finished with exit code 0

```

```

Run
AuthClient x AuthServer x
"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=58854" -Dfile.encoding=UTF-8 -Dsu
Enter Username: admin
Enter Password: 12345
Server Response: Login Successful
Process finished with exit code 0

```

3. Multi Service-Server

MultiServiceServer.java

```
package ServiceServer;
```

```
import java.io.*;
```

```
import java.net.*;
```

```

public class MultiServiceServer {
    public static void main(String[] args) {
        int port = 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started on port " + port);
            System.out.println("Waiting for connection...");

            Socket socket = serverSocket.accept();

```

```

System.out.println("Client connected!");

BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

String request;
while ((request = in.readLine()) != null) {
    if (request.equalsIgnoreCase("exit")) {
        System.out.println("Client exited.");
        out.println("Client exit.");
        break;
    }

    System.out.println("Client requested: " + request);

    String[] parts = request.split(" ", 2);
    String command = parts[0].toUpperCase();
    String text = parts.length > 1 ? parts[1] : "";

    String response;
    switch (command) {
        case "UPPER":
            response = text.toUpperCase();
            break;
        case "REVERSE":
            response = new StringBuilder(text).reverse().toString();
            break;
        case "COUNT":
            if (text.trim().isEmpty()) {
                response = "0 words";
            } else {
                int wordCount = text.trim().split("\\s+").length;
                response = wordCount + " words";
            }
            break;
        default:
            response = "Unknown command";
            break;
    }
}

```

```

        out.println(response);
    }

    socket.close();
    System.out.println("Client closed!");
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

MultiServiceClient.java

```

package ServiceServer;

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class MultiServiceClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 5000)) {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            while (true) {
                System.out.print("Enter service: ");
                String input = scanner.nextLine();

                if (input.trim().isEmpty()) continue;

                out.println(input);

                if (input.equalsIgnoreCase("exit")) {
                    String serverMsg = in.readLine();
                    if (serverMsg != null) {
                        System.out.println(serverMsg);
                    }
                    break;
                }
            }
        }
    }
}

```

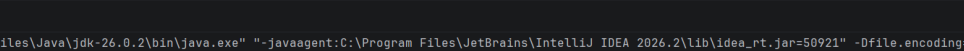


```
}

String response = in.readLine();
System.out.println("Server response: " + response);

catch (IOException e) {
    printStackTrace();
}
```

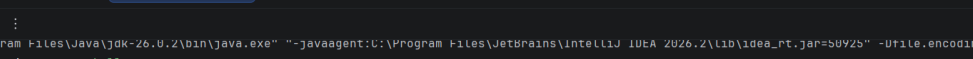
Output



```
Run MultiServiceServer x MultiServiceClient x
"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=50921" -Dfile.encoding=UTF-8 -Dsu
Server started on port 5000
Waiting for connection...
Client connected!
Client requested: upper hello
Client requested: reverse hello
Client requested: count hello world
Client exited.
Client closed!

Process finished with exit code 0

> src > ServiceServer > MultiServiceClient > main 9:49 (1154 chars, 34 line breaks) CRLF UTF-8 4 spaces
```



```
Run | MultiServiceServer | MultiServiceClient |  
"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=80925" -file.encoding=UTF-8 -Usu  
Enter service: upper hello  
Server response: HELLO  
Enter service: reverse hello  
Server response: olleh  
Enter service: count hello world  
Server response: 2 words  
Enter service: exit  
Client exit.  
  
Process finished with exit code 0  
  
src > ServiceServer > MultiServiceClient > main | 9:49 (1154 chars, 34 line breaks) CRLF UTF-8 4 spaces
```